

0.9.2. Tout l'alphabet, avec une seule boucle — doublé, deux positions

Tout l'alphabet deux fois : lignes 0 et 1, puis lignes 3 et 4.

1. Ce qu'il faut comprendre

C'est le 0.9 et le 0.9.1 réunis : l'alphabet du 0.9 (lignes 0 et 1), puis celui du 0.9.1 (lignes 3 et 4).

Les fonctions de cette leçon — et le tuto de chacune

- `#include <reste>` → [0.8.4. #include <reste> — a % b](#)
- `#include <diviser>` → [0.8.5. #include <diviser> — a / b](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **997 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
main()         123 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AdresseCase    10 o ← poser() - trouve l'adresse d'une case de l'écran
Diviser        17 o ← poser() - divise - le processeur ne sait pas le faire
Reste          10 o ← poser() - le reste d'une division
CopierTuiles   18 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles         704 o ← CopierTuiles - la police entière
```

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, EffacerCases, EcrireTexte, EcrireNombre, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
poser() × 2      80 o sur place, dans main() + AdresseCase (10 o), Diviser (17 o), Reste (10 o)
image() × 1      3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <reste>      // a % b, sauf par 1, 2, 4, 8, 16... écrits en clair
#include <diviser>     // a / b, sauf par 1, 2, 4, 8, 16... écrits en clair
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A
#include <poser>       // pose une tuile sur une case du fond
```

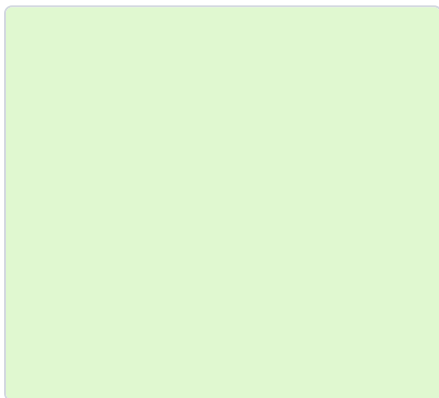
2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // 1. Lignes 0 et 1 (le 0.9)
    for (uint8_t i = 0; i < sizeof(ALPHABET); i++) {
        poser(i % 20, 0 + i / 20, ALPHABET[i]);    // 0 + i / 20 : la ligne 0, puis 1
    }
    // 2. Lignes 3 et 4 (le 0.9.1)
    for (uint8_t i = 0; i < sizeof(ALPHABET); i++) {
        poser(i % 20, 3 + i / 20, ALPHABET[i]);    // 3 + i / 20 : la ligne 3, puis 4
    }

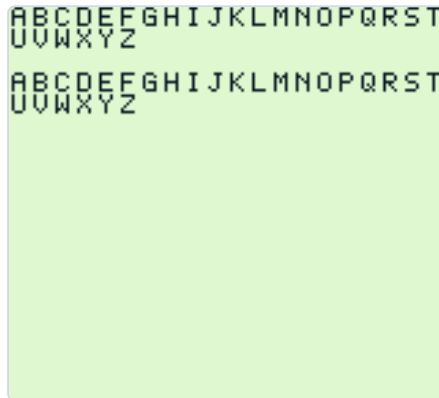
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Tout l'alphabet deux fois : en haut, et trois lignes plus bas.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ lignes 0 et 1

✓ lignes 3 et 4